SORTIRIA · RAPPORT DE CONDUITE DE PROJET DATA

Rapport de conduite de projet

Ce rapport raconte comment j'ai mené SortirIA, du besoin jusqu'au service en ligne. Il suit les cinq compétences évaluées : comprendre le besoin, auditer les données, choisir une solution, aider à décider, contrôler le projet.

Auteur : Stéphane Gonzalez **Formation** : Ingénieur IA, OpenClassrooms **Version** : 02/11/2026

Soutenance : 6/11/2026

Service : www.sortiria-montpellier.fr

Code : github.com/gonzalezcourrier34-glitch/SortirIA

00

Synthèse pour un décideur

Le problème. À Montpellier et sur le bassin de Thau, les sorties sont publiées dans dix endroits différents. Pour trouver un concert samedi soir, il faut ouvrir plusieurs sites et tout croiser à la main.

Ce que j'ai construit. SortirIA est un assistant. On écrit sa demande en français, par exemple « un concert petit budget samedi soir ». Il répond avec de vrais événements, à venir, et donne la source de chacun. Chaque nuit, un agent met la base à jour tout seul.

Où j'en suis. Le service est en ligne depuis le 15/10/2026. Tous les critères que j'avais fixés avant de mesurer sont atteints. Il coûte environ 10 $ par mois la première année.

Ce que l'usage réel a montré. Du 17 au 26/10, des testeurs ont posé 93 questions. Le service a tenu : 95 % des réponses sont arrivées en moins de 6,2 secondes, chaque question a reçu des propositions, et le service a continué de répondre pendant une panne de Mistral. Près d'une question sur deux (47 %) a trouvé une réponse exacte. Les autres portaient souvent sur des lieux ouverts toute l'année (cinéma, restaurants, marchés) : une piste claire pour enrichir la base.

Ma recommandation. Continuer, avec une nouvelle priorité. Le moteur est fiable et mesuré ; le prochain levier, c'est la couverture. Avant de réentraîner un modèle sur mes données (fine-tuning) ou de chercher des partenaires payants, j'ajouterai une base de lieux, puis je mesurerai la satisfaction avec des visiteurs venus d'eux-mêmes.

98,7 %

des propositions respectent date, lieu, public et budget (objectif : 95 %)

97,8 %

des 3 premières propositions sont pertinentes (objectif : 70 %)

6,4 s

95 % des réponses arrivent plus vite que ça (objectif : 8 s)

≈ 10 $

par mois ; 0,12 $ de Mistral pour 1 000 questions

01

Compétence : collecter les besoins métiers et analyser le contexte

Le besoin et le contexte

Le contexte

Je vis dans la région. Les informations sur les sorties existent, mais elles sont éparpillées : agendas des villes, offices de tourisme, OpenAgenda, programmes en PDF, sites des lieux.

Ce projet est personnel. J'ai donc joué deux rôles : celui qui exprime le besoin, et celui qui y répond. Pour aller au-delà de mon seul avis, j'ai confronté le besoin à d'autres personnes (voir plus bas).

Les utilisateurs

Qui	Ce qu'il veut	Ce que ça impose
Les habitants	Trouver une sortie pour ce soir ou ce week-end, en un seul endroit.	Des données fraîches, chaque jour.
Les touristes	Savoir ce qui se passe à Sète, Mèze ou Montpellier pendant leur séjour.	Couvrir deux zones : Montpellier et le bassin de Thau.
Les familles	Des sorties pour les enfants, gratuites ou sous un budget.	Le public et le prix sont des contraintes strictes, toujours respectées à la lettre.
Les lieux partenaires	Publier leurs offres simplement.	Accepter un fichier Excel, CSV, PDF ou Word.

Comment j'ai recueilli le besoin

- Mon propre usage : je suis l'utilisateur cible.
- Le projet et mon portfolio, présentés à mon mentor le 27/10. Ses retours m'ont fait mettre en avant les résultats chiffrés, et suivre chaque limite d'une piste concrète.
- Des tests avec des amis le 18/10. Ils m'ont demandé un choix « Tout public », un libellé « Les deux » plus clair que « Partout », et des offres partenaires dédoublonnées.
- Le journal de chaque question posée sur le site, et le pouce levé ou baissé sur chaque réponse.

La promesse, en une phrase

« Une personne décrit son envie en français. SortirIA lui propose des événements **réels, à venir, autour de Montpellier et du bassin de Thau**, qui **respectent sa demande** et **citent leur source**. »

Pour tenir cette promesse, il faut trois choses : des données fraîches, mises à jour toutes seules, des réponses justes et sourcées, et un service en production.

02

Compétence : auditer la solution data

L'audit des données

Avant de construire, j'ai regardé ce que valent les données disponibles. Je me suis posé quatre questions : ai-je le droit de les utiliser ? En ai-je assez ? Sont-elles fiables ? Que manque-t-il ?

Les sources et leur droit d'usage

Source	Droit d'usage	Volume	Ce que j'ai constaté
DATAtourisme (offices de tourisme)	Licence Ouverte 2.0	≈ 1 600 séances par nuit (1 612 le 28/10)	La plus riche. Peu de noms de lieux, horaires écrits en texte libre.
OpenAgenda	API ouverte, source citée	17 agendas utiles	Mon audit du 14/10 : la recherche par mots lisait 10 000 événements de 111 agendas pour en garder 83 dans la zone, en 6 minutes. J'ai donc choisi 12 agendas précis, puis 17 depuis le 18/10.
Programme PDF de Sète agglopôle	document public	434 événements	Une mise en page libre : un modèle de langage le lit pour en extraire les événements. Le site refuse les serveurs Azure : je le lis donc depuis mon poste.

Source	Droit d'usage	Volume	Ce que j'ai constaté
Fichiers des partenaires	fournis par le partenaire	exemples fictifs	Annoncés comme « Démonstration », en attendant les premiers partenaires.
Sites des lieux (lecture automatique, ou scraping)	seulement avec une autorisation écrite	0	J'ai demandé l'autorisation à Frontignan et à l'office de tourisme Archipel de Thau. J'attends leur réponse avant toute collecte.

Au total, la base contient environ **2 450 événements** sur les trois prochains mois.

La qualité des données

- **Des informations manquent souvent** : le prix, l'heure, l'âge. Ma règle : je garde l'information telle qu'elle est. Un prix inconnu s'affiche comme inconnu, bien distinct de « gratuit ».
- **Certaines fiches sortent du sujet** : offres d'emploi, collectes de sang. Elles sont rejetées avec un motif, compté chaque nuit.
- **Les sources se recoupent** : un même concert peut venir de deux sources. Je fusionne les doublons par titre, jour, heure et lieu.
- **Les sources tombent en panne** : le 14/10, une erreur de pagination a fait tomber DATAtourisme à 184 séances au lieu de près de 1 000. Depuis, une baisse anormale déclenche une alerte, et les événements de la veille sont gardés.
- **Le modèle peut se tromper** : chaque nuit, un second modèle relit et note 20 fiches tirées au hasard, comme un correcteur. Le score est passé de 89 % à 94 % après mes corrections.

Les données personnelles

Les visiteurs sont anonymes. Je garde uniquement l'empreinte d'un identifiant tiré au hasard. Le journal des questions est effacé après 90 jours.

Ma conclusion de l'audit

Les données ouvertes suffisent pour un premier service. Elles ont deux limites, et chacune ouvre une piste. Le bassin de Thau propose peu de sorties le soir : les programmes PDF et les partenaires y répondent. Et les lieux ouverts toute l'année (piscine, bowling) relèvent d'un annuaire de lieux : c'est la prochaine évolution de SortirIA, aujourd'hui centré sur les événements.

La solution technique

J'ai réutilisé des briques de mes projets de formation, puis je les ai adaptées. Pour chaque choix, je me suis posé une question simple : **le résultat peut-il être vérifié par du code ?** Si oui, j'écris une règle. Quand il faut comprendre un texte, j'utilise un modèle, et une règle contrôle ce qu'il rend.

Le fonctionnement en deux temps

- **La nuit**, un agent, c'est-à-dire un programme qui enchaîne seul les étapes (construit avec LangGraph), collecte les sources, vérifie, dédoublonne, enrichit avec Mistral et enregistre dans la base PostgreSQL.
- **Le jour**, l'API comprend la question avec des règles, filtre en SQL, cherche par le sens et par les mots, puis Mistral rédige une réponse courte qui cite ses sources.
- **Chaque lundi**, l'agent relit les recherches qui ont fait chou blanc. Il me propose des corrections, et je décide.

Mes choix, et pourquoi

Besoin	Mon choix	Pourquoi	Ce que j'ai écarté
Collecter plusieurs sources	Le moteur de CheckIt.AI (projet 12)	Une nouvelle source = un simple fichier de configuration.	Un script par source
Lire un PDF, juger la qualité	Un agent LangGraph	Ces étapes demandent de comprendre un texte, au-delà d'une simple copie de données.	Un ETL classique (copie de données)
Comprendre la question	Des règles	100 % de compréhension sur mon jeu de test, en 1 ms, gratuitement.	Un petit modèle (Ministral)
Chercher	PostgreSQL + pgvector (son extension de recherche par le sens)	Filtres stricts et recherche par le sens dans la même requête.	FAISS (projet 6)
Rédiger	mistral-small	Bon marché, hébergé en Europe.	ministral-8b : 17 % moins cher, mais plus lent
Mettre en ligne	Azure, CI/CD GitHub Actions	Presque gratuit la première année ; l'API s'endort en l'absence de visiteurs.	Un serveur allumé en permanence

Besoin	Mon choix	Pourquoi	Ce que j'ai écarté
Surveiller	Mail chaque matin, tableaux de bord Grafana, MLflow, dérive suivie chaque nuit (PSI) et chaque mois (Evidently)	Je suis seul : le service doit me prévenir quand il a besoin de moi.	Prometheus (réservé à un trafic plus fort)

Les garde-fous contre les inventions

- Mistral reçoit uniquement les événements trouvés en base. Chaque citation doit renvoyer à l'un d'eux, sinon elle est retirée.
- Un résumé qui cite un nom absent de la source est effacé. Exemple réel : « Cyrano de Bergerac » pour un spectacle nommé « CYRANA ».
- Pour un PDF, le titre doit être présent dans la page lue. Sinon, l'événement est rejeté.
- Chaque modification de configuration passe par mon accord. L'agent rédige une proposition de modification (pull request) ; je la relis.

Le détail technique : **Architecture**, **Agent**, **Recherche**, **Modèles**.

04

Compétence : apporter un appui stratégique et méthodologique à la décision

L'aide à la décision

Pour décider, il faut des critères clairs. J'ai donc fixé mes seuils **avant** de mesurer. Sinon, on finit par choisir le seuil qui arrange.

Un MVP en trois colonnes

- **Indispensable** : le cœur du produit. Tout est fait.
- **Important** : fait aussi, un vrai plus pour le produit (l'agent qui lit les PDF, les partenaires, les avis).
- **Pour plus tard** : la météo, la base de lieux, le fine-tuning. La lecture automatique des sites, elle, attend les autorisations.

Les critères de réussite, et le résultat

Critère	Seuil fixé avant	Mesuré	Décision
Propositions qui respectent les contraintes	≥ 95 %	98,7 %	✅ go

Critère	Seuil fixé avant	Mesuré	Décision
Compréhension des questions	≥ 95 %	100 %	✅ go
Pertinence des 3 premières propositions	≥ 70 %	97,8 %	✅ go
Accord du juge Mistral avec moi	≥ 85 %, indice d'accord AC1 ≥ 0,6	98,6 % ; 91,9 % sur des cartes nouvelles	✅ le juge peut rejouer la mesure
Temps de réponse (95 % des réponses)	≤ 8 s	6,0 à 6,4 s	✅ go
Coût Mistral	≤ 1 $ pour 1 000 questions	0,12 $	✅ go
Qualité des fiches (juge de la nuit)	≥ 80 %	89 à 94 %	✅ go
Réponses fidèles à la base (100 % d'événements réels)	100 %	100 %	✅ go

Comment j'ai rendu la mesure fiable

Le respect des contraintes se vérifie par le code. La pertinence, c'est un jugement : j'ai donc d'abord annoté moi-même 139 cartes. Ensuite seulement, j'ai réglé un juge Mistral sur mon annotation. Il a fallu 7 versions de sa consigne pour passer de 68 % à 98,6 % d'accord. J'ai appris qu'un modèle calcule mal les dates : j'ai confié au code ce qu'il vérifie mieux. Puis je l'ai testé sur 135 cartes toutes nouvelles pour lui : 91,9 %.

Les scénarios de coût

Usage par mois	1 000 questions	10 000 questions	100 000 questions
Coût la 1re année	≈ 10 $	≈ 17 $	≈ 32 à 38 $
Coût ensuite (base payante)	≈ 28 $	≈ 35 $	≈ 50 à 56 $

Une piste de recettes : un abonnement de 10 à 20 € par mois pour un lieu partenaire. Deux à quatre partenaires couvriraient le service. C'est une hypothèse, à confirmer par une étude de marché.

Les risques et mes parades

Risque	Probabilité	Parade
Une source change de format ou d'adresse	Forte	L'agent réessaie, garde la veille, diagnostique et propose une réparation.
Mistral indisponible	Moyenne (arrivé le 23/10)	Une réponse simplifiée, rédigée par le code et signalée à l'écran. Le 23/10 : 12 réponses servies ainsi. Depuis, un modèle de secours (mistral-medium) prend d'abord le relais, et chaque échec est journalisé avec sa cause.
Un coût qui dérape (robot)	Faible	1 000 recherches par heure au plus : au pire 85 à 145 $ de Mistral par mois.
Les demandes ou les sources changent	Moyenne	Rapport de dérive chaque nuit, alerte le lundi s'il est fort ; rapport Evidently complet chaque mois.
Peu de sorties le soir à Thau	Réelle	Alerte de couverture ; PDF, partenaires, agendas proposés par l'agent.

Le bilan de l'usage réel (17 au 26/10)

Mes jeux de test sont écrits par moi. J'avais donc prévu une phase avec des testeurs, et une décision sur trois chiffres : la part des pouces levés, la part des réponses exactes, et le nombre de partenaires intéressés. Les deux premiers se lisent dans le journal des questions, une fois mes propres questions de test mises de côté ; le troisième, dans mes échanges avec les villes :

Mesure	Résultat	Ce que j'en tire
Questions posées	93	Surtout le 18/10, jour des tests avec des amis ; l'usage retombe ensuite.
Réponse exacte	47 %	Les autres sont élargies. Les demandes ratées portent sur des lieux permanents : film au cinéma, horaires de restaurant, tapas, marché aux puces.
Questions avec au moins une proposition	100 %	L'élargissement propose toujours quelque chose.
Temps de réponse (95 % des réponses)	6,2 s	Sous l'objectif de 8 s, comme sur mon banc de test.
Coût Mistral	0,011 $ pour 93 questions	0,12 $ pour 1 000 questions, comme prévu.

Mesure	Résultat	Ce que j'en tire
Avis (pouces)	1, positif	Les testeurs ont préféré me faire leurs retours à l'oral. La prochaine phase mesurera la satisfaction.
Partenaires intéressés	réponse attendue	Sète et Frontignan démarchés ; j'attends leur réponse. Le circuit partenaire est prêt (fichier déposé, importé la nuit), montré avec des exemples.
Visiteurs distincts	comptés depuis le 27/10	J'ai ajouté au journal un identifiant anonymisé du navigateur, comme pour les avis : chaque visiteur est compté depuis.

L'usage réel confirme, en plus marqué, la limite mesurée hors ligne : 97,8 % de pertinence sur mes questions, 83,7 % sur des recherches courtes tirées au hasard, 47 % de réponses exactes sur de vraies questions. L'écart a la même cause : SortirIA est un agenda d'événements, et les gens cherchent aussi des lieux.

MA RECOMMANDATION

Go pour continuer, avec une priorité revue : la couverture avant le reste.

Le moteur tient ses engagements, même en production et même quand Mistral tombe. Le prochain levier, c'est ce que la base contient. Avant un fine-tuning ou des partenaires payants, je propose d'ajouter une base de lieux (piste 1 des évolutions), puis une nouvelle phase d'usage avec des visiteurs venus d'eux-mêmes, jugée sur trois chiffres : la part de réponses exactes, la part des pouces levés et le nombre de visiteurs qui reviennent.

05

Compétence : contrôler le projet (délais, coûts, livrables, performance)

Le contrôle du projet

Les délais

Date	Jalon	État
13/10/2026	Démarrage : reprise de mes projets précédents	✅

Date	Jalon	État
14/10/2026	Première collecte complète, schéma de la base, interface	✅
15/10/2026	Recherche évaluée, **mise en ligne sur Azure**	✅
16/10/2026	Déploiement automatique, pertinence mesurée, évaluation rejouée en CI	✅
17-19/10/2026	Retours des testeurs, incidents corrigés, suivi de la dérive	✅
17-26/10/2026	Usage réel avec des testeurs : 93 questions, 47 % de réponses exactes, usage arrêté après les tests	✅
27/10/2026	Bilan d'usage, recommandation revue ; visiteurs distincts comptés à partir de maintenant	✅
30/10-2/11/2026	800 recherches lancées en production, défauts trouvés et corrigés	✅
2/11/2026	Gel du code (étiquette v1.0-soutenance), deux jours avant la date prévue du 4/11	✅
6/11/2026	Soutenance	à venir

J'ai mis le service en ligne en trois jours. Depuis, j'ajoute uniquement ce que demandent les retours et les incidents (suivi de la dérive, modèle de secours) ; le reste du temps, je mesure, je fiabilise et je documente. C'est ce qui m'a permis de figer le code le 2/11, deux jours avant la date prévue.

Les coûts

- Environ **10 $ par mois** aujourd'hui : 9 $ pour l'interface, le reste est dans les offres gratuites d'Azure.
- Mistral : environ 0,005 $ par nuit de collecte, et 0,000118 $ par question.
- Un budget Azure de 20 € par mois m'envoie un mail à 50, 80 et 100 %.
- Un budget de 0,50 $ par nuit pour Mistral : au-delà, une alerte part dans le mail du matin.

Les livrables

Livrable	Où
Le service en ligne	**www.sortiria-montpellier.fr**
Le code, avec environ 4 090 tests	**github.com/gonzalezcourrier34-glitch/SortirIA**
La documentation technique	**gonzalezcourrier34-glitch.github.io/SortirIA**
Le dossier du projet (ce site)	**13 pages, dont les procédures et le suivi des incidents**

Livrable	Où
Les mesures, rejouables	`benchmarks/`, MLflow, rapport de la CI

La performance, et comment je la surveille

- **À chaque modification** : les tests automatiques (la CI) rejouent les tests et l'évaluation de la recherche. Sous 95 %, le déploiement est bloqué.
- **Chaque nuit** : un mail me donne le statut de la collecte, les chiffres, les alertes et le coût.
- **Toutes les 3 heures** : GitHub vérifie que le site, l'API, la base et la collecte répondent.
- **Chaque lundi** : les recherches ratées, les pouces bas, et le rapport de dérive, qui mesure si les usages changent. Je compare les 7 derniers jours aux 28 précédents : ce que les gens demandent, ce que SortirIA répond, ce que les sources apportent.
- **Chaque mois** (le 1er) : un rapport Evidently complet compare le mois écoulé au précédent, sur les questions et sur les nuits de collecte. GitHub Actions ouvre le pare-feu de la base le temps du rapport, puis le referme ; Evidently tourne à part, en dehors du service. Comme le suivi de chaque nuit, il attend d'avoir assez de données pour conclure.
- **À chaque version** : les 8 consignes et la chaîne (modèles Mistral, versions des consignes, jeux d'évaluation) sont enregistrées dans les registres de MLflow, et la CI exige une nouvelle version pour toute consigne modifiée. Je sais quelle version a produit quelle réponse, et laquelle la précédait.
- **Dans Grafana** : trois tableaux (collecte, recherche, modèles et coûts).

Quand un incident arrive, je le note : ce que j'ai vu, la cause, la correction, et le test qui empêche qu'il revienne. 31 incidents sont tracés dans la page **Suivi**.

Ce que je ferais différemment

- Fixer mes jeux d'évaluation encore plus tôt.
- Mettre des testeurs devant le site dès le premier jour en ligne.
- Demander les autorisations de collecte plus tôt : les réponses prennent du temps.
- Vérifier dès la mise en ligne que le journal enregistre tout ce dont le bilan aura besoin : c'est ainsi que j'ai ajouté le comptage des visiteurs.

06

Les mots techniques, en clair

Pour que tout lecteur puisse suivre, voici les termes du rapport, en une phrase chacun.

Terme	En clair
Agent (LangGraph)	Un programme qui enchaîne seul plusieurs étapes : collecter, vérifier, corriger. LangGraph est l'outil qui l'organise.
Mistral	Le modèle de langage (une IA qui lit et écrit du texte) que j'utilise, hébergé en Europe.
PostgreSQL, pgvector	La base de données, et son extension qui permet de chercher par le sens d'une phrase, au-delà des mots exacts.
Juge (LLM)	Un modèle qui note les réponses d'un autre, comme un correcteur. Je l'ai réglé sur mes propres notes.
CI/CD	Les tests automatiques lancés à chaque modification du code, puis la mise en ligne automatique si tout est bon.
Temps de réponse (95 %)	Le temps sous lequel arrivent 95 réponses sur 100. Les spécialistes l'appellent le P95.
Dérive (PSI, Evidently)	La mesure des changements dans les usages ou les données, d'une semaine ou d'un mois à l'autre. PSI est l'indicateur, Evidently l'outil du rapport mensuel.
MLflow	Le registre qui garde la trace de chaque version : consignes, modèles, mesures.
Grafana	Les tableaux de bord qui affichent les chiffres du service.
Pull request	Une proposition de modification du code, que je relis avant de l'accepter.
MVP	Le produit minimum utile : la plus petite version qui rend déjà service.
Fine-tuning	Réentraîner un modèle sur ses propres données pour le spécialiser.